

RooTitanium — Password, Interceptor header X, Aggiornamento Chromium

Guida di consegna passo-passo per Claude Opus (continuazione del lavoro di Fable)

Progetto: mini-browser QtWebEngine per SailfishOS 5.1 · Engine attuale: qt6-qtwebengine 6.8.3 (Chromium 122) · 13 luglio 2026

In due righe. Tre lavori che **condividono un unico rebuild** del binario: fai tutto in una passata sola. (1) **Password** = mini-gestore nostro, cattura credenziali via URL-scheme handler C++ + store cifrato. (2) **Interceptor header** = classe C++ per riscrivere gli header HTTP (Sec-CH-UA brand "Google Chrome") — leva *residua e incerta* per il login X. (3) **Aggiornamento Chromium** = il più grosso e il più vincolato: dipende dallo stack Qt6 community, non solo da noi. Ordine consigliato in fondo.

Perché un solo rebuild. Tutte e tre toccano C++ e/o il build di qt6-qtwebengine. Ogni ricompilazione del binario webengine-smoke costa (moc + g++ via sb2), e l'aggiornamento Chromium costa *ore*. Conviene: prima decidere se aggiornare Chromium (cambia l'albero sorgente sotto i piedi di tutto il resto), poi scrivere l'interceptor e il back-end Password nello stesso `main.cpp`, e ricompilare una volta.

0. Contesto minimo dell'architettura

Tutto il browser vive in pochi file. Ti serve solo questo per lavorare senza sorprese.

File	Ruolo
<code>smoke-test/test.qml</code>	Tutta la UI e la logica (~2170 righe). Qui vivono i due <code>WebEngineProfile</code> (id <code>normalProfile</code> / <code>incognitoProfile</code>), gli <code>userScript</code> di fingerprint, la pagina <code>settings.local</code> , e i rami di <code>onNavigationRequested</code> per le sotto-pagine interne <code>*.local</code> .
<code>smoke-test/main.cpp</code>	Launcher C++ + helper <code>rtNative</code> esposto al QML (DBus share, path PDF/Download, apri file). ~120 righe. Qui vanno l'URL-scheme handler (Password) e l'interceptor header.
<code>smoke-test/run.sh</code>	Avvio sul device: env Chromium, flag (touch-slop, device-scale-factor...), DevTools su 9222.
<code>packaging/qt6-qtwebengine-sfos/</code>	Spec, script di build (guardia VRM), RPMS. Serve solo per l'aggiornamento Chromium.

Ciclo di lavoro sul device

Device di test: `defaultuser@<IP-device>` (ssh e sudo senza password). Bundle in `~/webengine-smoke/`. Per provare una modifica al **solo QML**:

```
scp smoke-test/test.qml defaultuser@<IP>:/home/defaultuser/webengine-smoke/test.qml
# riavvio: MAI "pkill -f webengine-smoke" via ssh (uccide la shell). Kill via /proc:
ssh defaultuser@<IP> '
  for p in /proc/[0-9]*; do case "$(readlink $p/exe 2>/dev/null)" in *webengine-smoke*)
kill ${p#/proc/};; esac; done
  sleep 2; nohup /home/defaultuser/webengine-smoke/run.sh >/dev/null 2>&1 &'
```

Log runtime su device in `/tmp/rootitanium.log`. I `console.log` QML arrivano solo con `QT_FORCE_STDERR_LOGGING=1` (già in `run.sh`). Debug pagine: tunnel `ssh -L 9223:localhost:9222` → CDP su `http://localhost:9223`. La **sezione Permessi è già FATTA** (commit 664d74a): usala come **modello di riferimento** per il pattern "sotto-pagina di gestione `*.local` + sentinella in `onNavigationRequested`".

Ricompilare il binario (quando tocchi `main.cpp`). Il solo `scp` del QML non basta. Procedura (già collaudata, in memoria di progetto):

```
sfdk engine exec sb2 -t SailfishOS-5.1.0.11-aarch64.default
# moc del target: /usr/lib64/qt6/libexec/moc su main.cpp → main.moc
# g++ con: -I /usr/include/qt6/... e include WebEngine dal build tree:
#   packaging/qt6-qtwebengine-sfos/build/BUILD/qt6-qtwebengine-6.8.3/upstream/
include
# link: -L scratch/webengine-bundle/lib (le .so WebEngine bundle)
# poi scp del nuovo webengine-smoke nel bundle sul device.
```

Verifica che gli header `QWebEngineUrlScheme` / `QWebEngineUrlSchemeHandler` / `QWebEngineUrlRequestInterceptor` / `QWebEngineUrlRequestInfo` siano nell'include tree **PRIMA** di iniziare: sono tutti in **QtWebEngineCore**.

1. Sezione PASSWORD (richiede C++ + ricompilazione)

Verità architetturale, da leggere prima di scrivere codice. QtWebEngine **non espone alcun password manager al QML**. Il gestore interno di Chromium esiste ma è sepolto: l'unico aggancio nei sorgenti è `AutofillPopupController` in header privati, non istanziabile da QML. Quindi "salvare le password" qui significa **costruire un mini gestore nostro**: catturare le credenziali dai form, salvarle cifrate, e reinserirle al caricamento. Non è un toggle: è una funzionalità a sé.

Meccanismo di trasporto JS → app (la decisione chiave)

Per catturare user/password dal form serve che il JavaScript della pagina comunichi all'app. I canali possibili:

Canale	Giudizio
Navigazione a sentinella <code>*.local</code> (come le altre pagine)	NO: navigare via <code>location.href</code> sul submit abortisce il login. Da escludere.
Polling <code>runJavaScript</code> dal QML	Fragile per cogliere l'istante del submit; usalo solo per l'autofill (lettura/scrittura campi), non per la cattura.

Custom URL-scheme handler in C++ (QWebEngineUrlSchemeHandler), chiamato da JS con <code>fetch("rt-cred:save?...")</code>	Sì, consigliato. Un fetch verso uno schema custom viene intercettato in C++ senza bloccare la navigazione del form. Canale JS → C++ pulito e unidirezionale.
QWebChannel (<code>qwebchannel.js</code> + oggetto registrato)	Alternativa standard e bidirezionale, ma più infrastruttura. Valida se preferisci non toccare gli URL-scheme.

Consiglio: **URL-scheme handler** per la cattura (JS → C++) e **runJavaScript** per l'autofill (C++/QML → JS).

Passo 1 — Registrare lo schema e lo store in `main.cpp`

Prima di creare l'engine, registra lo schema (deve avvenire **prima** di `QtWebEngineQuick::initialize()`):

```
QWebEngineUrlScheme scheme("rt-cred");
scheme.setSyntax(QWebEngineUrlScheme::Syntax::Path);
scheme.setFlags(QWebEngineUrlScheme::SecureScheme |
QWebEngineUrlScheme::LocalAccessAllowed);
QWebEngineUrlScheme::registerScheme(scheme);
```

Poi installa un `QWebEngineUrlSchemeHandler` sui due profili (via `profile->installUrlSchemeHandler("rt-cred", handler)`). Il handler estrae i parametri dalla query, li passa allo store, e risponde con un 204/blob vuoto. Metti store e cifratura dentro `rtNative` (già esposto al QML) o in una classe gemella: così il QML può leggere l'elenco per la pagina di gestione.

Nota wiring profili: i due `WebEngineProfile` sono dichiarati in QML. `installUrlSchemeHandler` e `setUrlRequestInterceptor` (sezione 2) sono API C++ sull'oggetto profilo. Vedi il riquadro "wiring profili C+ ↔ QML" alla fine della sezione 2: vale per entrambe le feature.

Passo 2 — Schema SQLite

Puoi riusare lo stesso DB del resto (tabella nuova) oppure un file dedicato. Campi minimi:

```
CREATE TABLE IF NOT EXISTS passwords(
  origin TEXT NOT NULL,      -- es. https://x.com
  username TEXT NOT NULL,
  password BLOB NOT NULL,    -- CIFRATA, mai in chiaro
  ts INTEGER,
  PRIMARY KEY(origin, username)
);
```

Passo 3 — Cattura credenziali (user script)

Aggiungi uno script iniettato (come `rtDnt` / `rtYtTapFix` in `buildScripts()`) che sull'evento `submit` di qualunque form individua i campi password e il probabile username, e li invia:

```
document.addEventListener('submit', function(e){
  var f = e.target; if (!f || !f.querySelector) return;
  var pw = f.querySelector('input[type=password]'); if (!pw || !pw.value) return;
```

```
// username: il primo input testuale/email prima della password (euristica)
var user = '';
var inputs = f.querySelectorAll('input');
for (var i=0;i<inputs.length;i++){ var t=inputs[i]; if (t===pw) break;
    if (/text|email|tel/.test(t.type)) user = t.value; }
var q = 'o='+encodeURIComponent(location.origin)+'&u='+encodeURIComponent(user)
    + '&p='+encodeURIComponent(pw.value);
    fetch('rt-cred:save?' + q).catch(function({})); // intercettato in C++, non blocca il
submit
}, true);
```

Euristiche da rifinire: form multi-step (username e password in pagine separate, come proprio X), campi `autocomplete="new-password"` (registrazione, non da salvare come login), più form nella pagina. Per il primo taglio va bene la versione semplice; annota i limiti.

Passo 4 — Cifratura a riposo

Onestà sulla sicurezza. Salvare password è dato sensibile. Il minimo accettabile è cifrarle a riposo (non in chiaro nel DB). Con solo Qt a disposizione, la via pragmatica è una chiave derivata da un segreto del device (es. machine-id) usata con un cifrario simmetrico. **Ma** ciò protegge solo da chi legge il file, non da chi esegue codice come l'utente. La soluzione robusta su SailfishOS è il framework *Sailfish Secrets* o una **master password** chiesta all'utente (chiave mai su disco). **Decidi il livello con l'utente prima di promettere "sicuro"**; per un primo taglio dichiara esplicitamente che è "offuscamento locale", non cassaforte.

Passo 5 — Autofill al caricamento

Su `onLoadingChanged` con `LoadSucceededStatus` (dove già si registra la cronologia), interroga lo store per `origin` corrente; se c'è una credenziale, inietta con `runJavaScript` riempiendo i campi (**senza** inviare il form — l'utente conferma). Offri anche un banner "Salvare la password?" la prima volta (può essere un toast QML dopo la cattura, con "Salva"/"No").

Passo 6 — Sotto-pagina di gestione `passwords.local`

Come per i Permessi (già fatti): la riga "Password" in `settingsHtml()` apre `https://passwords.local/`; `passwordsHtml()` elenca origin + username (**mai** mostrare la password; al più un pulsante "mostra" dietro conferma), con azione elimina (sentinella che cancella la riga). Modella su `permissionsHtml()`: registra `passwords.local` come host interno nello stesso `onNavigationRequested`, accanto a `permissions.local`. La riga segnaposto da sostituire in `settingsHtml()` è:

```
<div class="srow dis"><span class="sbody"><span class="st">Password</span></span><span class="badge">In arrivo</span></div>
```

2. Interceptor header C++ per X (richiede C++ + ricompilazione)

Aspettative oneste, da leggere prima. La diagnosi definitiva del bug login X (in memoria di progetto) dice che il blocco *non* è (da solo) l'identità del browser: X ci tratta normalmente finché non si autentica un account reale, e il blocco scatta su `begin_login` = decisione reputazionale/anti-

automazione lato X. L'esperimento "Firefox mode" (UA Firefox completo) **non ha spostato nulla**. Quindi l'interceptor Sec-CH-UA è una leva **residua e a bassa probabilità**. La includiamo perché: (a) stiamo già ricompilando per la Password, quindi il costo marginale è basso; (b) è l'**unica via** per un'identità HTTP pulita al 100% (il brand "Google Chrome" invece di "Chromium", che gli anti-bot diffidano); (c) risolve una vera incoerenza rimasta. Ma **non promettere all'utente che sblocca X**.

Il problema preciso

Dai QML abbiamo già allineato lo UA, `navigator.userAgentData` (high-entropy Client Hints JS), `window.chrome` e `Accept-Language`. Resta scoperto il terzetto **Sec-CH-UA** low-entropy negli header HTTP:

```
Sec-CH-UA: "Chromium";v="122", "Not:A-Brand";v="24"      <- manca "Google Chrome"
Sec-CH-UA-Mobile: ?1
Sec-CH-UA-Platform: "Android"
```

Questi header li aggiunge il network stack di Chromium in C++ e **Qt non li espone al QML** (`WebEngineClientHints` tocca solo la parte high-entropy JS). Un vero Chrome manda brand `"Google Chrome";v="122"`; noi mandiamo `"Chromium"`. Un `QWebEngineUrlRequestInterceptor` in C++ è l'unico punto dove riscriverli.

Passo 1 — La classe interceptor

```
#include <QWebEngineUrlRequestInterceptor>
#include <QWebEngineUrlRequestInfo>

class HeaderInterceptor : public QWebEngineUrlRequestInterceptor
{
    Q_OBJECT
public:
    using QWebEngineUrlRequestInterceptor::QWebEngineUrlRequestInterceptor;
    void interceptRequest(QWebEngineUrlRequestInfo &info) override
    {
        // brand "Google Chrome" coerente con l'engine reale (122)
        info.setHttpHeader("sec-ch-ua",
            "\"Google Chrome\";v=\"122\", \"Chromium\";v=\"122\", \"Not:A-Brand\";v=\"24\"");
        info.setHttpHeader("sec-ch-ua-mobile", "?1");
        info.setHttpHeader("sec-ch-ua-platform", "\"Android\"");
        // opzionale: DNT reale come header (oggi è solo JS)
        // if (dntEnabled) info.setHttpHeader("dnt", "1");
    }
};
```

Passo 2 — Installarlo sui profili

```
// per-profilo (Qt6): vale per tutte le richieste di quel profilo
normalProfile->setUrlRequestInterceptor(new HeaderInterceptor(normalProfile));
incognitoProfile->setUrlRequestInterceptor(new HeaderInterceptor(incognitoProfile));
```

Rischio tecnico da verificare per primo (blocca tutto il resto). Storicamente `QWebEngineUrlRequestInfo::setHTTPHeader()` **non** riesce a sovrascrivere gli header che il network stack di Chromium gestisce da sé — `User-Agent` è il caso noto (per quello si usa `httpUserAgent` sul profilo, non l'interceptor). I **Sec-CH-UA** sono aggiunti da Chromium DOPO l'interceptor, quindi `setHTTPHeader` potrebbe essere **ignorato o accodato** anziché sostituire. **Prima di scrivere il resto, verifica empiricamente via CDP** (o `httpbin.org/headers`) che il valore inviato sia davvero quello riscritto. Se Chromium lo sovrascrive:

- opzione B: patchare il default brand nel build di Chromium (`embedder_support / user_agent` — dove si compone lo UA e i client hint low-entropy). Richiede rebuild del motore, si combina con la sezione 3.
- opzione C: flag Chromium in `run.sh` (es. disattivare Client Hints via `--disable-features=...`) — ma toglierli del tutto è un altro segnale-bot, quindi sconsigliato.

Wiring profili C++ ↔ QML (vale anche per la Password). I due profili sono dichiarati in QML (`WebEngineProfile { id: normalProfile ... }`) e le API `setUrlRequestInterceptor` / `installUrlSchemeHandler` sono C++. Il QML `WebEngineProfile` **non le espone**. Due strade, da scegliere a build-time:

1. **Spostare la creazione dei profili in C++** (in `main.cpp`) ed esporli al QML. Nota: il `WebEngineView.profile` del QML si aspetta un `QQuickWebEngineProfile`, non un `QWebEngineProfile` — **verifica in Qt 6.8 la compatibilità del tipo** prima di impegnarti su questa via.
2. **Ponte via `rtNative`**: un metodo C++ che riceve l'oggetto profilo QML e ci installa handler/interceptor accedendo al `QWebEngineProfile` sottostante. Anche qui va verificato in 6.8 come ottenere il `QWebEngineProfile*` da un `QQuickWebEngineProfile*`.

Questo dettaglio di API è l'unica incognita di integrazione: **risolvilo con una prova minima prima** di costruire Password e interceptor sopra.

3. Aggiornamento del motore Chromium (il più grosso — giorni, non ore)

Il vincolo che decide tutto. `QtWebEngine 6.N` richiede la **Qt base 6.N** identica sul device. Il device gira lo stack Qt6 **community di piggz** (OBS `home:piggz:qt6sb2`), oggi 6.8.x → per questo abbiamo buildato `webengine 6.8.3`. **Aggiornare Chromium = aggiornare l'intero stack Qt6 del device** (`qtbase`, `qtdeclarative`, ecc.) alla stessa minor. Quindi il tetto reale **non lo decidiamo noi**: lo decide quale versione Qt6 piggz pubblica per SFOS. Prima cosa da fare: **controllare che versione Qt6 base offre piggz** e allinearsi a quella.

Mappa Qt ↔ Chromium (per orientarsi)

Qt	Chromium	Note
----	----------	------

6.8.x (attuale)	122	Ciò che abbiamo buildato. (La riga <code>Provides: bundled(chromium)=118</code> nello spec è un residuo stantio dello spec chum: la 6.8.3 upstream è Chromium 122.)
6.9.x	130	Salto moderato.
6.10.x	134	Più recente stabile al cutoff.

Verifica sempre il numero reale nel README.chromium del tarball scelto: la mappatura cambia a ogni release. Non aggiornare "all'ultima" a prescindere: aggiorna alla minor **che piggy supporta sul device**.

Perché è il lavoro più costoso

- **Nuovo albero sorgente Chromium** = i ~10 fix di build che abbiamo dovuto applicare per la 6.8.3 (tetto 4 GB qemu, V8 sandbox/pointer-compression, rsp path-length, crash node/qemu seriali, V4L2 OPRGB, sockaddr include, link con `--no-keep-memory`, shim `v8_context_snapshot` con qemu-10, permessi 000 qmlcachegen...) **vanno riportati e ri-verificati**: alcuni scompaiono, altri cambiano forma, altri nuovi appaiono. Sono tutti documentati nella memoria `ripresa-rootitanium-6lug` — **leggila prima**.
- **Build da ore** anche in incrementale a vuoto: si riparte da zero (codice nuovo). Usa **sb2/cross-gcc nativo** (non qemu) con la **guardia VRM** — è ciò che sblocca il tetto 4 GB e va 5–10× più veloce. Skill `/rilascia-rootitanium` e `/12core` incapsulano già la procedura.
- **Deps Qt6**: nuovi BuildRequires / macro potrebbero mancare in qt6sb2 (vedi NOTE.md: già successo con qt6-srpm-macros, quickcontrols2, snappy). Da colmare pacchettizzando o inlinando, come fatto per la 6.8.3.

Procedura consigliata

1. **Ricognizione**: che Qt6 base offre piggy oggi? Se è ancora 6.8, l'aggiornamento Chromium è **bloccato a monte** finché non pubblica 6.9/6.10 → in tal caso **fermati e riferiscilo all'utente** (non ha senso buildare un webengine 6.9 che non gira con la qtbase 6.8 del device).
2. Prendi lo spec aggiornato da `sailfishos-chum/qt6-qtwebengine` per la nuova minor e ri-applica l'overlay RooTitanium (vedi `packaging/qt6-qtwebengine-sfos/NOTE.md`: BR rimossi, snappy, macro inlinate).
3. Configure via sb2 nativo (script `configure-only.sh`), poi build con guardia VRM. Attacca i fix di build man mano che i blocchi si ripresentano; documenta ognuno in memoria.
4. Repack RPM (metodo `cmake --install` in DESTDIR → spec minimale, come per la 6.8.3), rigenera il bundle in `scratch/webengine-bundle`, ricompila `webengine-smoke` con i nuovi include, deploy e smoke-test su device (`chrome://gpu` per confermare l'accelerazione HW ancora attiva).

Raccomandazione realistica. Se lo stack piggy è ancora 6.8, **Chromium resta a 122 per ora**: fai Password + interceptor (che non dipendono dalla versione) e rimanda l'aggiornamento a quando piggy pubblica una minor più nuova. Se invece piggy ha già 6.9/6.10, pianifica l'aggiornamento come sessione dedicata a sé (una notte di build con guardia VRM), **non** insieme a Password/ interceptor.

4. Ordine consigliato e checklist

1. **Decidi Chromium PRIMA** di toccare C++: controlla la Qt6 base di piggz. Se non c'è una minor più nuova → Chromium fermo a 122, prosegui con 2 e 3 sotto. Se c'è → valuta se farlo ora (sessione dedicata) o dopo Password/interceptor.
2. **Verifica l'incognita interceptor** (Sec-CH-UA sovrascrivibile via `setHTTPHeader ? test CDP/httpbin`) e il **wiring profili C++ ↔ QML** con una prova minima. È il gate tecnico comune a Password e interceptor.
3. **Password**: concorda con l'utente il livello di sicurezza (offuscamento vs master password vs Sailfish Secrets) **prima** di scrivere codice. Poi: schema URL + handler in `main.cpp` → cattura → store cifrato → autofill → pagina `passwords.local` (modella sui Permessi già fatti). Testa ogni pezzo via CDP.
4. **Interceptor**: aggiungi la classe e installala sui profili, nello stesso rebuild della Password. Verifica via CDP che gli header inviati siano quelli riscritti. Non promettere lo sblocco di X.
5. **Un solo rebuild** del binario per Password + interceptor. Deploy, smoke-test.
6. Aggiorna la memoria di progetto e committa. **Mai push remoto manuale**: il repo è privato, il push si fa con la skill `/push-github-rootitanium`.

Nota sul bug login X (contesto, non parte di questo lavoro). Il login X **non** è risolto e la diagnosi (memoria `rootitanium-next-tasks`) indica una protezione account/ambiente lato X, non il fingerprint. L'interceptor della sezione 2 è la sola leva tecnica residua ma incerta: costruirlo **potrebbe non** sbloccare X. Tienilo separato dalle aspettative sulla Password.